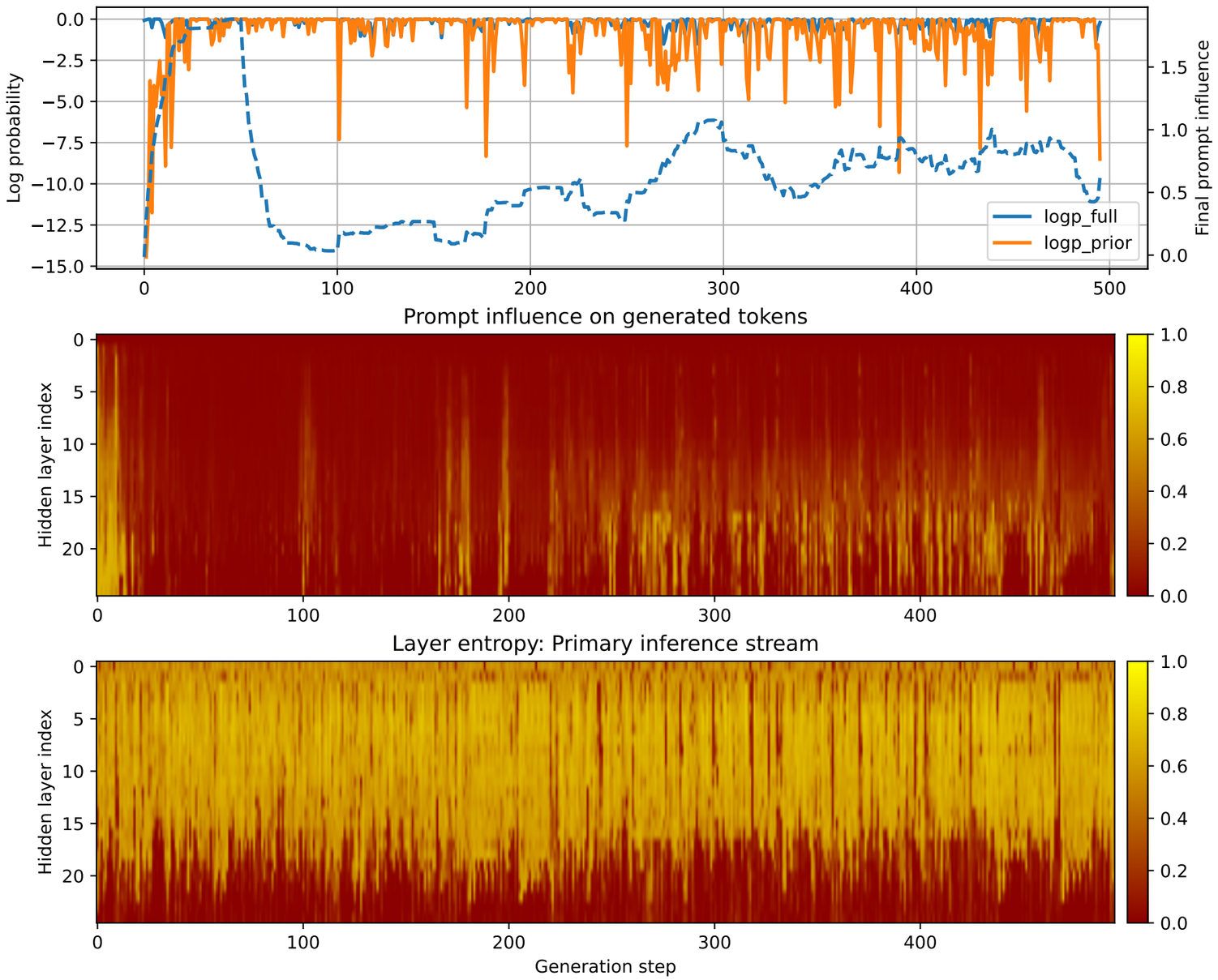
Prompt 1




```
PROMPT:
Task ID: PYC063  Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests.  Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC063  Your code must contain this required symbol:
def topological_sort  Implement this task:
define function topological_sort(graph) for DAG..  Start your
answer with the check-token line, then provide
the Python implementation.

MODEL RESPONSE:
# CHECK_TOKEN: PYC063  # Definition for a directed acyclic graph
(DAG) class DirectedAcyclicGraph:     def
__init__(self, vertices):         self.graph = {}         for
vertex in vertices:
self.graph[vertex] = []      def add_edge(self, u, v):         if
u not in self.graph or v not in self.graph:
self.graph[u] = [v]         else:
self.graph[u].append(v)      def topological_sort(self):
visited = set()         stack = [(self.graph, [])]          while
stack:             current_graph,
current_nodes = stack.pop()              for node in
current_graph:                 if node not in visited:
visited.add(node)                     stack.append((node,
current_nodes + [node]))          return
list(visited)  # Example usage graph = DirectedAcyclicGraph([1,
2, 3, 4, 5]) print(topological_sort(graph))  #
Output: [1, 2, 3, 4, 5]  # Explanation: - The `topological_sort`
function takes a directed acyclic graph (DAG)
as input. - It initializes an empty set `visited` to keep track
of visited nodes and a stack `stack` with the
initial state as a tuple containing the graph and an empty list
`current_nodes`. - The `while` loop runs as
long as the stack is not empty. - In each iteration of the loop,
it pops the topmost tuple from the stack and
unpacks it into `current_graph` and `current_nodes`. - For each
node in `current_graph`, it checks if it has
already been visited by adding it to the `visited` set and
appending its list of neighbors to the
`current_nodes` list. - If the node has not been visited, it is
added to the `visited` set and the stack is
pushed with the node and its list of neighbors. - After the loop
finishes, the `visited` set is converted to a
list and returned as the result.  In the example usage, we create
a DAG with 5 vertices (1, 2, 3, 4, 5) and
call the `topological_sort` function with this graph. The output
is `[1, 2, 3, 4, 5]`, which represents the
topological order of the graph.
```